VARDHAMAN COLLEGE OF ENGINEERING

(Autonomous)

Hyderabad, Telangana

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

A COURSE END PROJECT REPORT ON

PROCESS SCHEDULER IMPLEMENTATION

OPERATING SYSTEMS LABORATORY - A9510

Submitted by:

U.Nithin	25881A0529
W.Nitish	25881A0530
T.Pavan	25881A0531

Batch no. 10

Course Facilitator

Ms. Kavitha Chitrala

Assistant Professor, CSE



VARDHAMAN COLLEGE OF ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

2026–27

CERTIFICATE

This is to certify that the Course End Project titled **"PROCESS SCHEDULER IMPLEMENTATION"** is carried out by **"U.Nithin", "W.Nitish" and "T.Pavan "** with Roll Numbers **"25881A0529", "25881A0530" and "25881A0531"** towards the **A9510 – Operating Systems Laboratory** course in partial fulfilment of the requirements for the award of degree of Bachelor of Technology in Computer Science and Engineering during the Academic Year 2026–27.

Course Facilitator	**Academic Head**	**Head of the Department**
Ms. Kavitha Chitrala	Dr. Karthikeyan	Prof. Y. Vijaylatha
Assistant Professor, CSE	Professor, CSE	Department of CSE

Contents

Abstract 4

1 Introduction 4

2 Literature Review 5

3 Objectives 6

4 Concepts Used 6

4.1 Process Scheduling 6
4.2 Arrival Time 6
4.3 Burst Time 6
4.4 Priority 7
4.5 Preemptive Priority Scheduling 7
4.6 Round Robin Scheduling 7
4.7 Waiting Time 7
4.8 Response Time 7
4.9 Average Waiting Time 7
4.10 Average Response Time 7

5 Methodology 8

5.1 Architectural Diagram 8
5.2 Hardware and Software Requirements 9
5.3 Working Principle 10
5.4 Basic Algorithm 10
5.4.1 Algorithm 1: Preemptive Priority Scheduling 10
5.4.2 Algorithm 2: Round Robin Scheduling 11

6 CPU Scheduling Methods 17

6.1 Preemptive Priority Scheduling 17
6.2 Round Robin Scheduling 17
6.3 Performance Evaluation 18

7 Results and Discussion 18

7.1 Preemptive Priority Scheduling 18
7.2 Round Robin Scheduling 18
7.3 Discussion 20

8 Mapping of (POs) and SDGs **20**

8.1 Program Outcomes (POs) 20

8.2 Mapping of Sustainable Development Goals (SDGs) 20

9 Conclusion **21**

Abstract

Process scheduling is an important function of an Operating System that determines the order in which processes are executed by the CPU. Efficient scheduling helps improve CPU utilization, reduce process waiting time, and provide better responsiveness to users. This project, titled “Process Scheduler Implementation,” focuses on implementing Preemptive Priority Scheduling and Round Robin Scheduling using the C programming language with a Graphical User Interface (GUI).

The system accepts process information such as process ID, arrival time, burst time, and priority, along with the time quantum required for Round Robin Scheduling. In Preemptive Priority Scheduling, the CPU executes the highest-priority available process and may interrupt the currently running process when a higher-priority process arrives. In Round Robin Scheduling, each process receives a fixed CPU time quantum in a cyclic order.

The application calculates the waiting time and response time of each process and determines the average waiting time and average response time for the selected scheduling algorithm. The GUI provides an interface for entering process details and displaying scheduling results in an understandable format.

The project helps demonstrate the working principles of CPU scheduling algorithms and provides a practical way to compare their performance using calculated scheduling metrics.

Keywords: Operating System, Process Scheduling, C Programming, Graphical User Interface, Preemptive Priority Scheduling, Round Robin Scheduling, Waiting Time, Response Time.

1 Introduction

An Operating System is system software that manages computer hardware and software resources. One of its important responsibilities is process management, which includes creating, scheduling, executing, and terminating processes. Since multiple processes may compete for CPU time, the Operating System uses CPU scheduling algorithms to decide which process should execute next.

Process scheduling plays an important role in determining system performance. Different scheduling algorithms use different rules to allocate CPU time. Their performance can be evaluated using parameters such as waiting time, response time, turnaround time, and CPU utilization.

Preemptive Priority Scheduling selects a ready process according to its priority. If a process with a higher priority arrives while another process is executing, the CPU can be allocated to the newly arrived process. This approach allows important processes to

receive CPU time earlier, although lower-priority processes may experience starvation.

Round Robin Scheduling allocates a fixed time quantum to each ready process in a cyclic order. If a process does not complete within its allotted quantum, it returns to the ready queue and waits for another turn. This method is commonly associated with time-sharing systems because it provides processes with repeated opportunities to execute.

The project implements both scheduling algorithms using the C programming language and a Graphical User Interface. Users can enter process information, select a scheduling algorithm, provide the required parameters, and view the calculated scheduling results.

The application is intended to improve understanding of CPU scheduling through practical implementation and comparison of average waiting time and average response time.

2 Literature Review

CPU scheduling is a fundamental concept in Operating Systems. It determines how the CPU is allocated among processes in the ready queue. Different scheduling algorithms are designed to meet different performance requirements, including responsiveness, fairness, and reduced waiting time.

Priority Scheduling assigns a priority to each process and selects a process according to its priority level. In Preemptive Priority Scheduling, the currently executing process can be interrupted when a higher-priority process becomes ready. The performance of this algorithm depends on the priority assignment and the arrival and burst times of the processes. Starvation may occur when low-priority processes are repeatedly postponed.

Round Robin Scheduling is designed to share CPU time among processes. Each process receives a fixed time quantum, after which an unfinished process is placed at the end of the ready queue. The choice of time quantum influences the number of context switches and the responsiveness of the system.

Waiting time measures the total time a process spends waiting in the ready queue. Response time measures the interval between a process's arrival and its first CPU execution. These metrics help evaluate the performance of scheduling algorithms from different perspectives.

A GUI-based scheduling application can make these concepts easier to understand by accepting process parameters and presenting calculated results. The proposed project combines both scheduling algorithms in a C program with a graphical interface to support practical learning and performance comparison.

3 Objectives

The main objectives of this project are:

1. To understand the fundamentals of CPU scheduling in Operating Systems.
2. To implement Preemptive Priority Scheduling using the C programming language.
3. To implement Round Robin Scheduling using the C programming language.
4. To develop a Graphical User Interface for entering process information and displaying results.
5. To accept process ID, arrival time, burst time, and priority as input parameters.
6. To accept the time quantum required for Round Robin Scheduling.
7. To calculate the waiting time and response time of individual processes.
8. To calculate the average waiting time and average response time for both algorithms.
9. To compare the behaviour of the two scheduling algorithms using their calculated results.
10. To improve understanding of process execution, preemption, and time-sharing.

4 Concepts Used

4.1 Process Scheduling

Process scheduling is the activity of selecting a process from the ready queue for CPU execution. The scheduling algorithm determines which process executes and when the CPU is allocated to it.

4.2 Arrival Time

Arrival time is the time at which a process enters the ready queue and becomes available for execution.

4.3 Burst Time

Burst time is the total CPU execution time required by a process to complete its execution.

4.4 Priority

Priority indicates the relative importance assigned to a process. In this project, a smaller priority number represents a higher priority.

4.5 Preemptive Priority Scheduling

Preemptive Priority Scheduling selects the highest-priority process among the available ready processes. If a higher-priority process arrives during execution, the currently running process can be interrupted.

4.6 Round Robin Scheduling

Round Robin Scheduling allocates a fixed time quantum to each process in the ready queue. If a process does not complete within its allotted time, it returns to the end of the queue.

4.7 Waiting Time

Waiting time is the total time a process spends waiting in the ready queue.

$$WT_i = CT_i - AT_i - BT_i$$

4.8 Response Time

Response time is the time between a process's arrival and its first CPU execution.

$$RT_i = ST_i - AT_i$$

4.9 Average Waiting Time

$$AWT = \frac{\sum_{i=1}^{n} WT_i}{n}$$

4.10 Average Response Time

$$ART = \frac{\sum_{i=1}^{n} RT_i}{n}$$

5 Methodology

5.1 Architectural Diagram

The architecture of the proposed Process Scheduler Implementation consists of an input module, a scheduling algorithm module, a calculation module, and an output module. The application allows the user to select Preemptive Priority Scheduling or Round Robin Scheduling and displays the calculated results through a GUI.

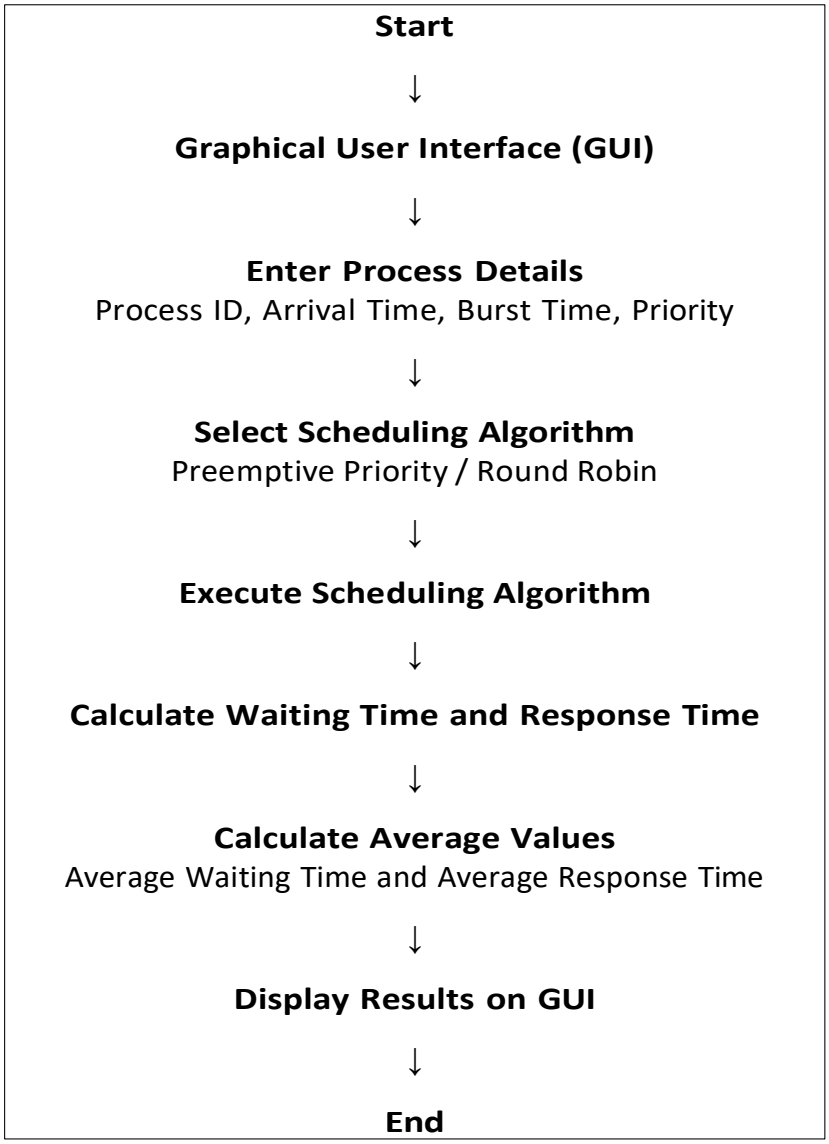


Figure 1: Architecture of the Process Scheduler Implementation

The GUI accepts the required process information and scheduling parameters. The scheduling module executes the selected algorithm, while the calculation module determines the waiting time and response time for each process. The output module displays the individual process results and the average values.

5.2 Hardware and Software Requirements

Hardware Requirements:

- Computer or laptop for program development and execution.
- Keyboard and display for entering process information and viewing scheduling results.
- Sufficient memory and processing capability to execute the C program and GUI.

Software Requirements:

- C programming language.
- C compiler such as GCC or another compatible compiler.
- GUI library compatible with the selected C development environment.
- Code editor or Integrated Development Environment (IDE).
- Operating System that supports the selected compiler and GUI library.

Input Requirements:

- Number of processes.
- Process ID for each process.
- Arrival time of each process.
- Burst time of each process.
- Priority of each process for Preemptive Priority Scheduling.
- Time quantum for Round Robin Scheduling.

Output Requirements:

- Process-wise completion time.
- Process-wise waiting time.
- Process-wise response time.
- Average waiting time.
- Average response time.
- Scheduling results displayed through the GUI.

The selected GUI library and development environment determine the exact compilation and execution requirements. The program must validate the inputs before executing the selected scheduling algorithm.

5.3 Working Principle

The Process Scheduler Implementation operates by accepting process information through a graphical user interface and executing the selected CPU scheduling algorithm.

1. **Input:** The user enters the number of processes, arrival times, burst times, and process priorities.
2. **Algorithm Selection:** The user selects either Preemptive Priority Scheduling or Round Robin Scheduling.
3. **Parameter Configuration:** For Round Robin Scheduling, the user specifies a positive time quantum.
4. **Scheduling:** The program determines the order of CPU execution according to the selected algorithm.
5. **Time Recording:** The program records the first execution time and completion time of each process.
6. **Calculation:** The program calculates the waiting time and response time of each process.
7. **Average Calculation:** The average waiting time and average response time are calculated for all processes.
8. **Output:** The GUI displays the process-wise results and the calculated average values.

The correctness of the results depends on the scheduling logic, proper handling of process arrival times, and accurate recording of the first CPU execution time.

5.4 Basic Algorithm

The Process Scheduler Implementation consists of two CPU scheduling algorithms: Preemptive Priority Scheduling and Round Robin Scheduling.

5.4.1 Algorithm 1: Preemptive Priority Scheduling

1. Start the program.
2. Read the number of processes.
3. Read the process ID, arrival time, burst time, and priority of each process.
4. Initialize the remaining burst time of every process.

5. Set the current time to the earliest process arrival time.
6. Identify all processes that have arrived and are not completed.
7. Select the available process with the highest priority.
8. If no process is available, advance the current time to the next process arrival.
9. Record the first execution time when a process executes for the first time.
10. Execute the selected process for one time unit.
11. Update its remaining burst time.
12. Repeat the selection process at the next scheduling decision.
13. When a process completes, record its completion time.
14. Continue until all processes are completed.
15. Calculate the waiting time and response time of each process.
16. Calculate and display the average waiting time and average response time.
17. Stop the program.

5.4.2 Algorithm 2: Round Robin Scheduling

1. Start the program.
2. Read the number of processes.
3. Read the process ID, arrival time, and burst time of each process.
4. Read the time quantum.
5. Initialize the remaining burst time of every process.
6. Initialize the ready queue.
7. Add the processes that have arrived to the ready queue.
8. Select the process at the front of the ready queue.
9. Record its first execution time if it has not executed before.
10. Execute the process for the smaller of the time quantum and its remaining burst time.
11. Update the remaining burst time and current time.

12. Add newly arrived processes to the ready queue.
13. If the process is unfinished, place it at the end of the queue.
14. If the process completes, record its completion time.
15. Repeat until all processes are completed.
16. Calculate the waiting time and response time of each process.
17. Calculate and display the average waiting time and average response time.
18. Stop the program.

```
The program is implemented using the C programming language.
It provides two CPU scheduling algorithms: Preemptive Priority
Scheduling and Round Robin Scheduling.

The user enters the number of processes, arrival time, burst
time, and priority of each process. The program then executes
the selected scheduling algorithm and calculates the completion
time, waiting time, and response time of every process.

In Preemptive Priority Scheduling, the process with the highest
priority is selected from the available processes. In this
implementation, a smaller priority number represents a higher
priority.

In Round Robin Scheduling, each process receives CPU time
according to a fixed time quantum. If a process does not
complete within its allocated time, it is placed at the end
of the ready queue.

Finally, the program calculates and displays the average waiting
time and average response time for the selected algorithm.
\usepackage{listings}
\usepackage{xcolor}


#include <stdio.h>

#define MAX 100

```

```
struct Process {
    int id, at, bt, pr, rem;
    int ct, wt, rt, first;
};

void display(struct Process p[], int n) {
    float awt = 0, art = 0;
    int i;

    printf("\nPID\tAT\tBT\tPR\tCT\tWT\tRT\n");

    for (i = 0; i < n; i++) {
        p[i].wt = p[i].ct - p[i].at - p[i].bt;
        p[i].rt = p[i].first - p[i].at;

        awt += p[i].wt;
        art += p[i].rt;

        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
               p[i].id, p[i].at, p[i].bt, p[i].pr,
               p[i].ct, p[i].wt, p[i].rt);
    }

    printf("\nAverage Waiting Time = %.2f\n", awt / n);
    printf("Average Response Time = %.2f\n", art / n);
}

void priority(struct Process p[], int n) {
    int time = 0, done = 0, i, k;

    for (i = 0; i < n; i++) {
        p[i].rem = p[i].bt;
        p[i].first = -1;
    }

    while (done < n) {
        k = -1;

        for (i = 0; i < n; i++) {
            if (p[i].at <= time && p[i].rem > 0) {
                if (k == -1 || p[i].pr < p[k].pr)
```

```
                    k = i;
                }
            }

            if (k == -1) {
                time++;
                continue;
            }

            if (p[k].first == -1)
                p[k].first = time;

            p[k].rem--;
            time++;

            if (p[k].rem == 0) {
                p[k].ct = time;
                done++;
            }
        }

        display(p, n);
}

void roundRobin(struct Process p[], int n, int q) {
        int time = 0, done = 0, i, k;
        int queue[MAX * MAX], front = 0, rear = 0;
        int added[MAX] = {0};

        for (i = 0; i < n; i++) {
            p[i].rem = p[i].bt;
            p[i].first = -1;
        }

        while (done < n) {
            for (i = 0; i < n; i++) {
                if (!added[i] && p[i].at <= time) {
                    queue[rear++] = i;
                    added[i] = 1;
                }
```

```
112                 }
```

```
113
114         if (front == rear) {
115             time++;
116             continue;
117         }
118
119         k = queue[front++];
120
121         if (p[k].first == -1)
122             p[k].first = time;
123
124         int run = p[k].rem < q ? p[k].rem : q;
125
126         for (i = 0; i < run; i++) {
127             time++;
128             p[k].rem--;
129
130             int j;
131             for (j = 0; j < n; j++) {
132                 if (!added[j] && p[j].at <= time) {
133                     queue[rear++] = j;
134                     added[j] = 1;
135                 }
136             }
137         }
138
139         if (p[k].rem == 0) {
140             p[k].ct = time;
141             done++;
142         } else {
143             queue[rear++] = k;
144         }
145     }
146
147     display(p, n);
148 }
149
150 int main() {
151     struct Process p[MAX];
152     int n, i, choice, q;
153
```

```
154     printf("Enter number of processes: ");
155     scanf("%d", &n);
156 
157     if (n < 1 || n > MAX) {
158         printf("Invalid number of processes\n");
159         return 0;
160     }
161 
162     for (i = 0; i < n; i++) {
163         p[i].id = i + 1;
164 
165         printf("Enter arrival time, burst time and priority  for P
              %d: ",
166                 i + 1);
167         scanf("%d%d%d", &p[i].at, &p[i].bt, &p[i].pr);
168 
169         if (p[i].at < 0 || p[i].bt <= 0 || p[i].pr < 0) {
170             printf("Invalid process details\n");
171             return 0;
172         }
173     }
174 
175     printf("\n1. Preemptive Priority Scheduling\n");
176     printf("2. Round Robin Scheduling\n");
177     printf("Enter choice: ");
178     scanf("%d", &choice);
179 
180     if (choice == 1) {
181         priority(p, n);
182     } else if (choice == 2) {
183         printf("Enter time quantum: ");
184         scanf("%d", &q);
185 
186         if (q <= 0) {
187             printf("Invalid time quantum\n");
188             return 0;
189         }
190 
191         roundRobin(p, n, q);
192     } else {
193         printf("Invalid choice\n");
```

```
194     }
195
196     return 0;
197 }
198
199 \usepackage{listings}
200 \usepackage{xcolor}
201
202 \lstset{
203     language=C,
204     basicstyle=\ttfamily\small,
205     numbers=left,
206     numberstyle=\tiny,
207     breaklines=true,
208     frame=single,
209     showstringspaces=false
210 }
```

6 CPU Scheduling Methods

6.1 Preemptive Priority Scheduling

In preemptive priority scheduling, each process is assigned a priority. The CPU executes the process with the highest priority. If a higher-priority process arrives, the currently running process is interrupted.

```
Process P1 (Priority 3) --> Running
Process P2 (Priority 1) --> Preempts P1
Process P2              --> Running
```

This method allows important processes to execute earlier but may cause starvation of low-priority processes.

6.2 Round Robin Scheduling

In Round Robin scheduling, each process is assigned a fixed time interval called the time quantum. Processes are executed one after another in a circular queue.

```
Ready Queue: P1 -> P2 -> P3
Time Quantum: 2 units


P1 -> P2 -> P3 -> P1 -> ...
```

This method provides fair CPU allocation to processes and is suitable for time-sharing systems.

6.3 Performance Evaluation

The performance of both scheduling algorithms is evaluated using waiting time and response time.

Waiting Time (WT) = Completion Time - Arrival Time - Burst Time

Response Time (RT) = First Execution Time - Arrival Time

Average WT = Total Waiting Time / Number of Processes

Average RT = Total Response Time / Number of Processes

These measures help compare the efficiency and responsiveness of the Preemptive Priority and Round Robin scheduling algorithms.

7 Results and Discussion

The developed CPU scheduling simulator was implemented using the C programming language. The program allows users to select Preemptive Priority Scheduling or Round Robin Scheduling and enter the required process details. The simulator calculates the completion time, waiting time, response time, average waiting time, and average response time.

7.1 Preemptive Priority Scheduling

In this test, four processes are entered with their arrival time, burst time, and priority. The simulator executes the Preemptive Priority Scheduling algorithm and displays the scheduling results for each process.

7.2 Round Robin Scheduling

In this test, four processes are entered with their arrival time and burst time. A time quantum of 2 units is used. The simulator executes the Round Robin Scheduling algorithm and displays the scheduling results.

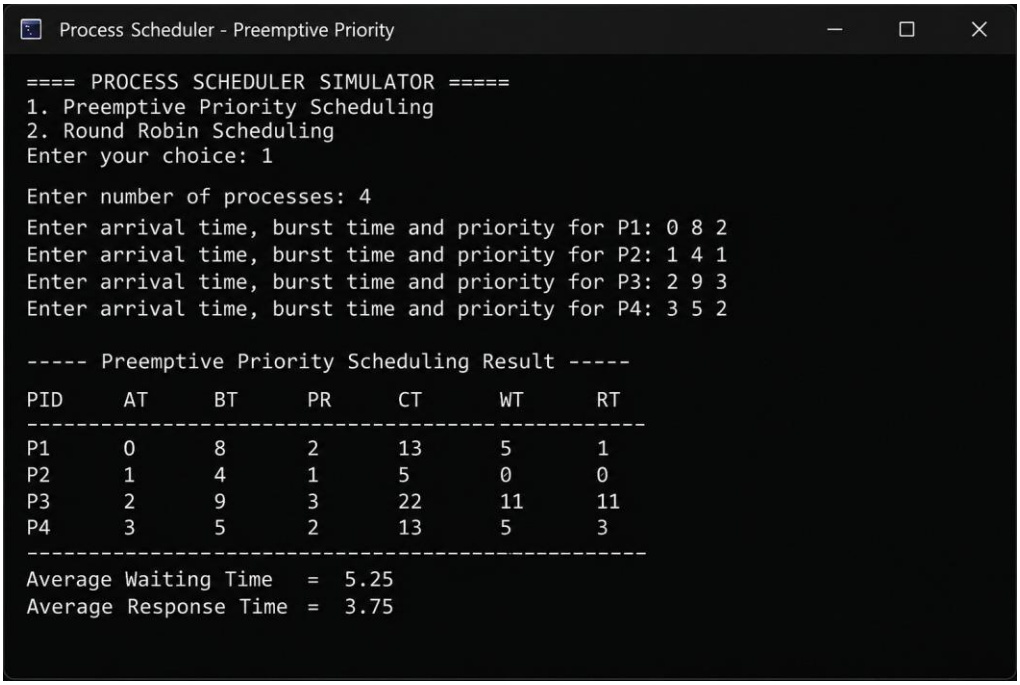


Figure 2: Output of Preemptive Priority Scheduling

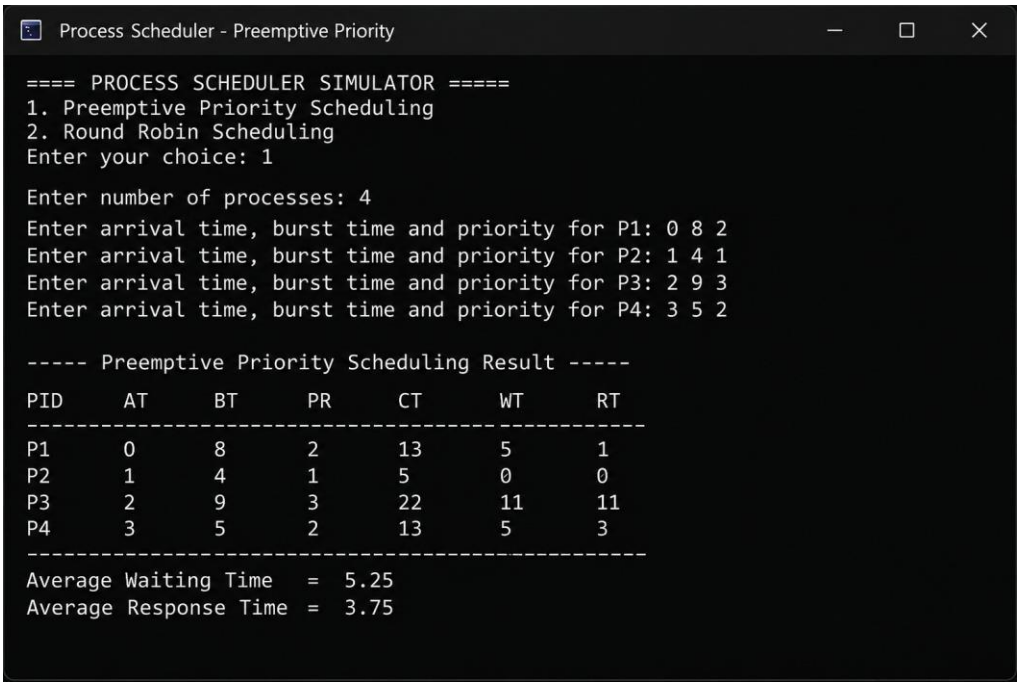


Figure 3: Output of Round Robin Scheduling

The output displays the completion time, waiting time, and response time of each process. The program also calculates the average waiting time and average response time for all processes.

7.3 Discussion

The results demonstrate how the two scheduling algorithms allocate CPU time to processes. Preemptive Priority Scheduling selects processes according to their priority, allowing higher priority processes to execute before lower-priority processes when required. However, low priority processes may experience longer waiting times.

Round Robin Scheduling allocates CPU time in fixed intervals and places unfinished processes at the end of the ready queue. This provides a fair opportunity for processes to execute and is suitable for time-sharing systems.

The average waiting time and average response time help evaluate the performance of both algorithms. The results depend on the arrival times, burst times, priority values, and time quantum supplied by the user.

8 Mapping of (POs) and SDGs

8.1 Program Outcomes (POs)

The Process Scheduler Implementation project helps develop programming, problemsolving, analytical, and software development skills. The following table shows the mapping of the project with relevant Program Outcomes (POs).

Mapping scale: 1 – Low, 2 – Medium, 3 – High. The mapping levels are proposed and should be confirmed with the project supervisor and department guidelines.

8.2 Mapping of Sustainable Development Goals (SDGs)

The Process Scheduler Implementation project supports efficient computing and software development practices. Its relevance to the Sustainable Development Goals (SDGs) is described below.

SDG 4: Quality Education

The project provides a practical learning platform for understanding operating system concepts, CPU scheduling algorithms, and performance evaluation. It supports technical education through hands-on implementation in C.

SDG 9: Industry, Innovation and Infrastructure

The project demonstrates the implementation of fundamental operating system scheduling algorithms. It encourages software innovation and helps students understand how computing systems manage processes and CPU resources efficiently.

Table 1: Mapping of Project with Program Outcomes

PO	Description	Mapping Level
PO1	Engineering knowledge and application of computing fundamentals.	3
PO2	Problem analysis and identification of suitable CPU scheduling solutions.	3
PO3	Design and development of a software solution for process scheduling.	3
PO4	Investigation and analysis of scheduling results using waiting and response times.	2
PO5	Use of modern programming tools and the C programming language.	3
PO9	Individual and teamwork skills during project development.	2
PO10	Communication of the project design, implementation, and results.	2
PO12	Independent learning of operating system scheduling concepts.	2

SDG 12: Responsible Consumption and Production

Efficient CPU scheduling can contribute to responsible use of computing resources by organizing process execution. However, this project does not directly measure energy consumption or environmental impact, so its contribution to this goal is indirect.

Table 2: Mapping of Project with Sustainable Development Goals

SDG	Goal	Project Contribution
SDG 4	Quality Education	Provides practical learning of CPU scheduling and operating system concepts.
SDG 9	Industry, Innovation and Infrastructure	Demonstrates software development and process management techniques.
SDG 12	Responsible Consumption and Production	Encourages awareness of efficient computing resource utilization.

9 Conclusion

The Process Scheduler Implementation project successfully demonstrates two CPU scheduling algorithms: Preemptive Priority Scheduling and Round Robin Scheduling using the C

programming language. The program accepts process details such as arrival time, burst time, and priority, and calculates the completion time, waiting time, and response time for each process.

Preemptive Priority Scheduling executes processes according to their priority, whereas Round Robin Scheduling allocates CPU time to each process using a fixed time quantum. The average waiting time and average response time help evaluate and compare the performance of these algorithms.

This project provides a practical understanding of CPU scheduling, process management, and operating system concepts. It also improves programming and problem-solving skills through the implementation of scheduling algorithms.

Future Work

The Process Scheduler Implementation project can be enhanced in several ways to improve its functionality, usability, and performance.

- **Graphical User Interface (GUI):** Develop a user-friendly graphical interface that allows users to enter process details and select scheduling algorithms easily.
- **Gantt Chart Visualization:** Display a Gantt chart to represent the execution order and time intervals of processes, making scheduling results easier to understand.
- **Additional Scheduling Algorithms:** Extend the program to support other CPU scheduling algorithms such as First Come First Serve (FCFS), Shortest Job First (SJF), and Shortest Remaining Time First (SRTF).
- **Performance Comparison:** Provide a comparison of average waiting time, average response time, and average turnaround time for different scheduling algorithms.
- **Dynamic Process Management:** Allow users to add processes dynamically and observe how new arrivals affect the scheduling order.
- **Improved User Input Validation:** Add input validation to handle invalid arrival times, burst times, priorities, and time quantum values.
- **Exporting Results:** Provide an option to save scheduling results in a text file or report for further analysis.

These enhancements can make the project more interactive, informative, and useful for understanding CPU scheduling techniques and their performance.

Bibliography

References

[1] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th Edition, Wiley, 2018.

[2] W. Stallings, *Operating Systems: Internals and Design Principles*, 9th Edition, Pearson, 2018.

[3] A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, 4th Edition, Pearson, 2015.

[4] B. W. Kernighan and D. M. Ritchie, *The C Programming Language*, 2nd Edition, Prentice Hall, 1988.

[5] GeeksforGeeks, "CPU Scheduling in Operating Systems," Available at: https://www.geeksforgeeks.org/ (Accessed: October 2026).